Synchronized 2D-3D IFC Bridge Viewer

Introduction & Context

Bridge inspections are undergoing a digital transformation. While the future lies in **3D BIM (IFC) Models**, many inspectors still rely on traditional **2D damage sketches** for their clarity and ease of use in the field.

To bridge this gap, we want to create a tool that offers the best of both worlds: a 3D model for spatial context and synchronized 2D "unfolded" views for detailed documentation.

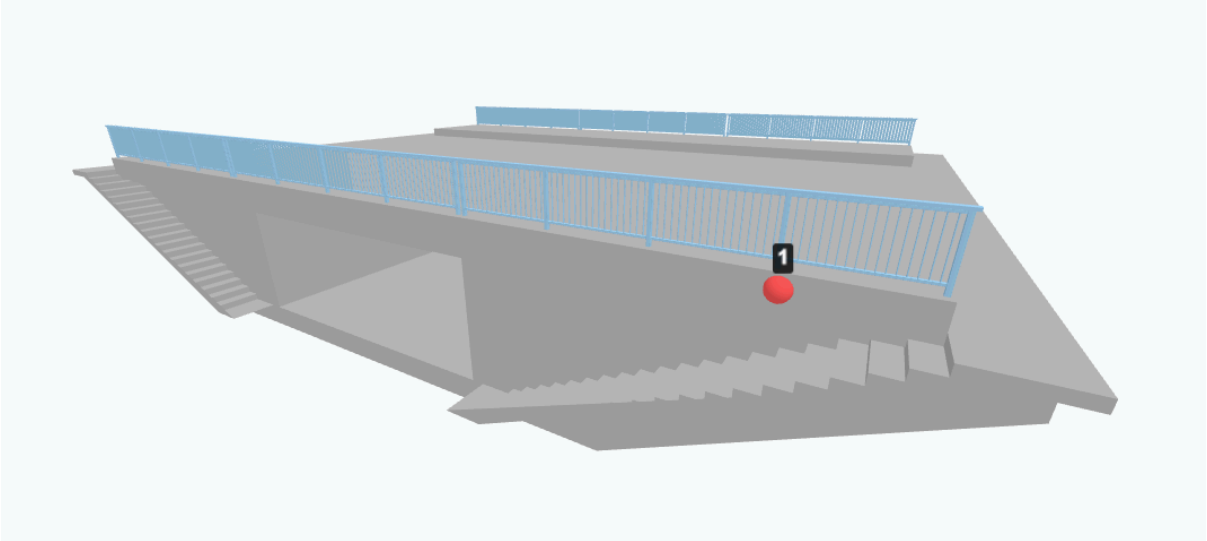
The Objective

Your task is to build a **React-based MVP** (Minimum Viable Product) that synchronizes a 3D IFC model with multiple 2D sectional views using the [That Open Engine]() (formerly IFC.js).

🛠️ Technical Requirements

1. The 3D View

- Load and display an IFC model (use the provided model).
- Implement a **Marker System**: Allow users to place 3D markers (e.g., small spheres) on the model surface to indicate damage.
- Markers should be **draggable** or movable in the 3D space.

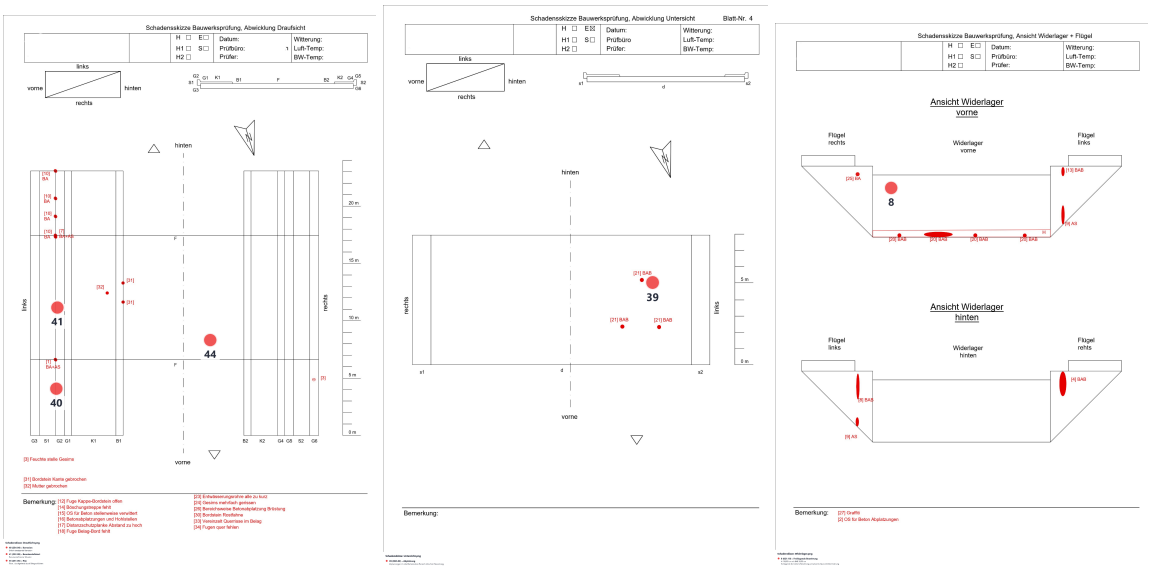


2. The 2D Damage Sketches

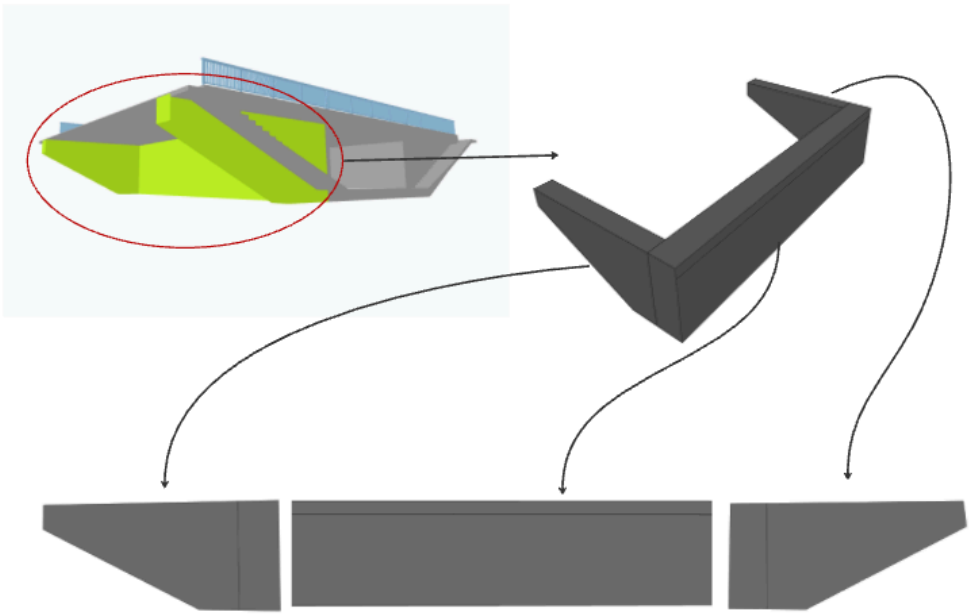
Inspectors use specific views. You need to generate:

- **Draufsicht (Top View):** A horizontal top-down cut (first sketch).
- **Untersicht (Soffit/Bottom View):** A horizontal cut looking up from the bottom (second sketch).

- **Side Views (Developed/Folded):** This is the "challenge" part. Here we have not just a side cut, but the side cut is folded open, having the abutment wall at the middle (which is the actual side), and the wing walls folded open and attached to it (third sketch).
  You may need to combine multiple clipping planes to create a "flattened" 2D representation.



Damage Sketches: Top View, Bottom View, Side Views



Explanation on how to fold the abutment and wing walls for the side view

3. Bidirectional Synchronization

- **3D-2D:** When a marker is placed or moved in the 3D viewer, its projection must appear instantly in the corresponding 2D view(s).
- **2D-3D:** Moving a marker in a 2D sketch should update its position in the 3D model.

4. Stack & Tools

- **Framework:** React + Typescript (functional components, hooks).
- **Engine:** That Open Engine (Components, Clipper, etc.).
- **Optional:** You are encouraged to use AI coding assistants (GitHub Copilot, Cursor, etc.). We value efficiency and your ability to leverage modern tools.

📋 Submission Guidelines

1. Code Quality:

We aren't looking for a 100% finished product, but we do look for **clean, readable, and modular code**. Follow React best practices (prop types, clean state management, Thinking in React – React).

2. Documentation:

Include a short `README.md` or a separate note answering:

- What was your exact setup? (Libraries, AI tools used).
- What worked well, and what were the main technical hurdles?
- How did you handle the "folded" 2D side views?

3. Communication:

If you get stuck or find that the task is taking much longer than expected, **please reach out**. Knowing when to ask for clarification or help is a senior trait we highly value. It won't negatively impact your application.

Estimated Effort: Roughly **one full working day**. If you reach a point where you've demonstrated the core logic (the "MVP"), feel free to stop and document your progress. The goal is by no means to develop a complete solution. Take shortcuts when it makes sense to do so.

💡 Tips for Success

- Check out the **Clipper**, **Edges, and Raycaster** components in the That Open Engine documentation for creating the 2D views.
- For the 2D-3D sync, consider how you map 3D world coordinates to the local 2D plane coordinates of your "cuts."

We look forward to seeing your approach! Happy coding.